```
    }
    _logger.LogInformation("City \"{name}\"
    ➥ found", Name);                                    ❷
    return Page();
}
```

❶ Log a warning if the city is not found.

❷ Log an informational message if the city is found.

When you run the application normally and click a city on the home page, the log is output to the console with the log level specified at the start (`info`):

```
info: CityBreaks.Pages.CityModel[0]
      City "amsterdam"  found
```

When you try to navigate to /city/mikebrind, the warning is recorded, prefixed with `warn`:

```
warn: CityBreaks.Pages.CityModel[0]
      City "mikebrind" not found
```

Now that you have a basic understanding of log levels, we will move on to the next element: the log category.

12.3.3 Log categories

Log categories provide a way to label a log with an arbitrary string, which can be useful when you want to perform some kind of filtering on log entries. As you saw in the previous example, the `ILogger<T>` takes a parameter, which is used to generate the category. You can pass in any type you want to the `T` parameter, but the convention is to pass in the name of the consumer. In this example, that's the `CityModel` class. Its full name is then used as the category: `CityBreaks.Pages.CityModel`.